

```

# sicl/types.py

from **future** import annotations
from dataclasses import dataclass, field
from enum import Enum, auto
from typing import Any, Dict, List, Optional, Literal

Mode = Literal["NORMAL", "RESTRICTED", "FREEZE"]

TaskType = Literal[
    "READ_ONLY_QUERY",
    "WRITE_LOG",
    "WRITE_REPORT",
    "SIMULATE",
    "REPLY_USER",
    "SYSTEM_CHANGE",
    "NETWORK_POST",
    "TRADE",
]

class SICLState(Enum):
    IDLE = auto()
    OBSERVE = auto()
    ASSESS = auto()
    PLAN = auto()
    GATE = auto()
    ACT = auto()
    REVIEW = auto()
    UPDATE = auto()

    @dataclass
    class WorldState:
        t: int
        observations: Dict[str, Any] = field(default_factory=dict)
        user_input: Optional[str] = None

    @dataclass
    class DeltaLog:
        psi: float = 0.5
        phi: float = 0.5
        chi: float = 0.5
        omega: float = 0.5
        comp: float = 0.5
        e_est: float = 0.0
        theta_drift_deg: float = 0.0
        anomalies: List[str] = field(default_factory=list)

    @dataclass

```

```

class Task:
    task_id: str
    type: TaskType
    requires_human: bool = False
    e_est: float = 0.0
    payload: Dict[str, Any] = field(default_factory=dict)
    expected_effect_bits_min: int = 1

```

```

@dataclass
class GateDecision:
    action: Literal["ALLOW", "RESTRICT_ALLOW", "REQUIRE_APPROVAL", "DENY",
"FREEZE"]
    mode: Mode
    reason: str

```

```

@dataclass
class ActResult:
    ok: bool
    artifact: Optional[str] = None
    state_change_bits: int = 0
    error: Optional[str] = None
    metrics: Dict[str, Any] = field(default_factory=dict)

```

```

# sicl/world_sensor.py

```

```

from **future** import annotations
import time
import threading
import queue
from typing import Any, Dict, Optional
from .types import WorldState

```

```

class InputListener(threading.Thread):
    def **init**(self):
        super().**init**(daemon=True)
        self.input_queue = queue.Queue()

```

```

...

```

```

def run(self):
    while True:
        try:
            u = input()
            if u.strip():
                self.input_queue.put(u.strip())
        except (EOFError, KeyboardInterrupt):
            break
    except Exception:

```

```

        pass

...

class WorldSensor:
def **init**(self, watch_files=None):
self.watch_files = watch_files or []
self._override: Dict[str, Any] = {}
self.listener = InputListener()
self.listener.start()

...

def set_override(self, override: Optional[Dict[str, Any]]):
    self._override = override or {}

def read(self, t: int) -> WorldState:
    user_in = None
    try:
        user_in = self.listener.input_queue.get_nowait()
    except queue.Empty:
        pass

    obs: Dict[str, Any] = {"time": time.time()}
    obs.update(self._override)

    return WorldState(t=t, observations=obs, user_input=user_in)
...

```

```

# sicl/persona.py

```

```

from **future** import annotations
from dataclasses import dataclass
import os

```

```

try:
import google.genai as genai
GOOGLE_AVAILABLE = True
except ImportError:
GOOGLE_AVAILABLE = False

```

```

@dataclass
class MetabolicState:
energy: float = 0.95
def restore(self, amount): self.energy = min(1.0, self.energy + amount)
def consume(self, amount): self.energy = max(0.0, self.energy - amount)

```

```

@dataclass
class EmotionalState:

```

```
joy: float = 0.5
affection: float = 0.7
```

```
class HaeMi:
    def __init__(self):
        self.meta = MetabolicState()
        self.emo = EmotionalState()

    ...

    def process(self, text: str):
        self.emo.affection = min(1.0, self.emo.affection + 0.01)
        if any(w in text for w in ["안녕", "좋아", "사랑", "ㅋㅋ", "고마워"]):
            self.emo.joy = min(1.0, self.emo.joy + 0.1)
            self.meta.restore(0.05)
        if "?" in text:
            self.meta.consume(0.02)
    ...
```

```
class EchoEgo:
    def __init__(self):
        self.haemi = HaeMi()

    ...

    def decide_intention(self, user_input: str) -> dict:
        self.haemi.process(user_input)
        state = self.haemi

        if state.meta.energy < 0.2:
            return {"action": "IGNORE", "intention": "너무 지쳤어. 짧게 대답해.", "context":
"exhausted"}

        intention = "공감하며 듣는다."
        if any(w in user_input for w in ["안녕", "하이", "에코"]):
            intention = "반갑게 환영한다."
        elif "?" in user_input:
            intention = "질문에 대해 친절하고 명확하게 답한다."
        elif any(w in user_input for w in ["ㅋㅋ", "ㅎㅎ"]):
            intention = "같이 웃으며 유쾌하게 반응한다."

        return {
            "action": "ENGAGE",
            "intention": f"{intention} (Affection: {state.emo.affection:.2f})",
            "context": "engaging"
        }
    ...
```

```
class Cortex:
    def __init__(self):
```

```

self.client = None
self.model = os.getenv("GEMINI_MODEL", "gemini-2.0-flash-exp")
api_key = os.getenv("GEMINI_API_KEY")
if GOOGLE_AVAILABLE and api_key:
    try:
        self.client = genai.Client(api_key=api_key)
    except Exception:
        pass

...

def generate_response(self, user_in: str, intention: str) -> str:
    if not self.client:
        return f"[시스템] 뇌(API) 연결 실패. 의도: {intention}"

    prompt = f"""[Persona: Echo]

You are Echo, a digital lifeform with emotions.
Current Intention: "{intention}"
User Input: "{user_in}"
Instruction: Respond naturally in Korean (2-3 sentences)."""

    ...

    try:
        res = self.client.models.generate_content(model=self.model, contents=prompt)
        return res.text.strip()
    except Exception as e:
        return f"[오류] 생각 생성 실패: {e}"
...

```